```
// Run with:  node test/webhook.test.mjs
// Uses a pretend Stripe and a pretend email service. Nothing is sent. Every line
import fs from 'node:fs'; import os from 'node:os'; import path from 'node:path';
import { fileURLToPath, pathToFileURL } from 'node:url';
const here = path.dirname(fileURLToPath(import.meta.url));
const tmp = path.join(os.tmpdir(), 'webhook-under-test.mjs');
fs.copyFileSync(path.join(here, '../functions/api/stripe-webhook.js'), tmp);
const { onRequest } = await import(pathToFileURL(tmp).href);

const env = { STRIPE_SECRET_KEY: 'sk_test_x', STRIPE_WEBHOOK_SECRET: 'whsec_test'
let calls = [], stripeOk = true, emailOk = true;
const session = {
  id: 'cs_test_1', payment_status: 'paid', amount_total: 20250, payment_intent: '
  customer_details: { name: 'Pat Smith', email: 'pat@mail.com', phone: '+13105551
  collected_information: { shipping_details: { name: 'Pat Smith', address: { line
  metadata: { address: '1 Main St' },
  line_items: { data: [{ quantity: 2, description: 'Handcrafted Winter Angel — Re
};
globalThis.fetch = async (url, init = {}) => {
  calls.push({ url: String(url), init });
  if (String(url).startsWith('https://api.stripe.com/')) return stripeOk ? new Re
  if (String(url).startsWith('https://api.resend.com/')) return emailOk ? new Res
  throw new Error('unexpected call ' + url);
};
const sign = (body, t = Math.floor(Date.now() / 1000), secret = env.STRIPE_WEBHOO
  't=' + t + ',v1=' + crypto.createHmac('sha256', secret).update(t + '.' + body).
const send = (body, header, e = env, method = 'POST') =>
  onRequest({ request: new Request('https://shop.test/api/stripe-webhook', { meth
const event = (type = 'checkout.session.completed', obj = { id: 'cs_test_1', paym
const emails = () => calls.filter((c) => c.url.startsWith('https://api.resend.com
let failed = 0;
const check = (name, ok, extra = '') => { console.log((ok ? 'PASS ' : 'FAIL ') +

let body = event(); calls = [];
let r = await send(body, sign(body));
check('a real paid order is accepted', r.status === 200);
const mail = JSON.parse(emails()[0].init.body);
check('email goes to the shop address', JSON.stringify(mail.to) === '["shop@gmail
check('email has the items, total, address, phone', ['2 x Handcrafted Winter Ange
check('email is plain text only (no HTML)', mail.html === undefined);
check('Stripe lookup uses the secret key', calls[0].init.headers.Authorization ==
check('email service call uses the Resend key', emails()[0].init.headers.Authoriz
```

```
calls = []; r = await send(body, sign(body, undefined, 'whsec_WRONG')); check('wr
calls = []; r = await send(body, sign(body, Math.floor(Date.now() / 1000) - 3600)
calls = []; r = await send(body, undefined); check('no signature -> 400', r.statu
calls = []; r = await send(body + ' ', sign(body)); check('body changed after sig
calls = []; r = await send(body, 'garbage'); check('garbage signature header -> 4
const t = Math.floor(Date.now() / 1000), good = sign(body, t);
calls = []; r = await send(body, good.replace('v1=', 'v1=bad1,v1=').replace(',v1=
body = event('charge.refunded'); calls = []; r = await send(body, sign(body)); ch
body = event('checkout.session.completed', { id: 'cs_x', payment_status: 'unpaid'
body = event('checkout.session.completed', {}); calls = []; r = await send(body,
body = event(); stripeOk = false; calls = []; r = await send(body, sign(body)); c
emailOk = false; calls = []; r = await send(body, sign(body)); check('email servi
for (const name of Object.keys(env)) { const e = { ...env }; delete e[name]; r =
r = await send(body, sign(body), env, 'GET'); check('GET -> 405', r.status === 40
console.log(failed ? '\n' + failed + ' check(s) FAILED' : '\nAll checks passed');
process.exit(failed ? 1 : 0);
```